```
        This function returns a vector of features, f, for the given state-
action pair

        Compute: f = [f_1(s, a), f_2(s, a), ... , f_n(s, a)]

        """
```

Once you have determined the number of features you wish to define, assign the integer value to the "num_features" variable in Play Ground and Evaluation cells.

In addition to writing the get_features() function you will also implement the get_Q_value() and the update() function of the LearningAgent class.

```
def get_Q_value(self, state, action):
        """
        Description
        ----------
        This function returns the Q value; Q(state, action) = weightVector
. featureVector
        where . is the dotProduct operator

        Compute: Q(s, a) = w . f = w_1 * f_1 + w_2 * f_2 + ... + w_n * f_n

        """
```

```
def update(self, state, action, next_state, reward):
        """
        Description
        ----------
        This function updates the weights based on the given transition

        Compute:
```

| Sections you will edit: | |
|---|---|
| Learning Agent | This section implements the class outline for theLearning agent. This is the section where you will implement your code. Look for "*** YOUR CODE HERE ***" |

| | |
|---|---|
| | You may add new helper functions if needed. |
| | |
| **Sections you should read but NOT edit:** | |
| Environment | This section defines the autonomous driving environment and |

We are putting our best efforts in testing the source code. However, should the code break due to a bug in the source code, we would encourage you to modify the necessary lines in the source code and include a README cell within your notebook explaining the bug and your fix.

**Evaluation:** Your code will be autograded for technical correctness. Please do not change the names of any provided functions or classes within the code, or you will wreak havoc on the autograder. However, the correctness of your implementation -- not the autograder's judgements -- will be the final judge of your score. If necessary, we will review and grade assignments individually to ensure that you receive due credit for your work.

**Grading:** Each question will be evaluated over a certain number of test cases. If your agent passes at least 70% cases, you will be awarded full points for the question. However, note that we may introduce new test cases (other than the ones in the Test Cases section) when grading your submissions. Thus, as a developer, it is your responsibility to test your agent in various unique scenarios before turning in your code.

| | | |
|---|---|---|
| | | initialized at the top of the grid. Default is 0.2. |
| --seed | Integer | This seed is used to populate other cars on the road randomly. A given seed will result in the same initialization of other cars on the road. Default is 3. |
| --episodes | Integer | Specify the number of episodes you wish to train the agent on. This argument is used by the learning agent only. Default is 50000. |
| [IMPORTANT] --num_features | Integer | Specify the number of features you wish to define for your learning agent in the "Grade Q3" cell by assigning the integer value to "num_features". If this argument is not passed and there is a mismatch in the number of features defined, the code will throw an error. Default is 1. |

**Submission**

After working on your code, follow the following steps.

- Save your FinalExam_PartB.ipynb notebook file to your drive.
- Exit from the lockdown browser and end your quiz.
- Upload and submit your FinalExam_PartB.ipynb to the canvas assignment titled "Final Exam PartB Submission" within 5 minutes of ending the quiz. If your submission time exceeds this window, we will not accept your submission.

In the event that something went wrong with our source code and you had to edit other files provided, please be sure to include a README cell in your notebook explaining the issue and changes made to the code.

When you make multiple submissions, Canvas may append a number to your submission name (e.g., FinalExam_PartB-1.ipynb), which is fine.

## Libraries

This project uses numpy, random, and copy libraries that are not available by default in a python installation. You may have to install these libraries prior to beginning your project.

## Visualization

The autonomous driving environment is a grid-based discretized simulation of driving on a highway as shown below.

to None. Also, no new cars are deployed in the environment once the environment is initialized.
At any step, if there is a crash or if the AA has successfully crossed the scope of the road, the simulator terminates and displays the respective message.

**Getting Started**

Run all the cells in the following sections:

Environment
State
Agent
Problem

Exploring the environment through Manual Agent
To get started, run the Autonomous Driving simulator in manual control mode found in the section "Playground - Manual Agent" which uses keyboard inputs.

The commands available for manual control are:
F: to move forward
L: to move left

| | |
|---|---|
| Problem | This section implements the controllers for different agent types. |
| Playground | This section initializes different agent types for you to play around with the code. |
| Evaluation | This is the section that grades your agent program. |
| | |
| **Sections you can ignore:** | |
| Test Cases | This section has sample test cases encoded for your project. |

**Sections to Edit and Submit:** You will write your code in the LearningAgent class found in the Agent section. Please do not change the other sections in this notebook. After writing your code, download your notebook file (.ipynb), and upload it on canvas.

**What should you do if there is a bug in the source code?**

The autonomous agent (AA) is indicated by the character 'A'.
The other cars on the road are indicated by the character '^'.

The direction of movement of all cars is from the bottom of the grid to the top of the grid. The goal is to have the AA drive successfully across the end of the road without crashing into other cars or moving out of the road (which is also considered as a crash).

**General Information on the Simulator**

At the start of the program, the environment is initialized with an autonomous driving agent (referred to as AA throughout this project) and some other cars at random locations on the road.

The position of a car on the grid is represented by (r, c) where, r indicates the row (counting from 0, starting from the bottom to the top of the grid) and c indicates the column of the grid (counting from 0, starting from the left to the right of the grid).

At every step, the AA and other cars take an action from the list of available actions: move forward (F), move left (L), move right (R), and wait (W). The list of available actions are all considered legal for the AA at any given step. However, the legal actions for any other car depends on its surroundings at a given step. The movement of the other cars (i.e. all cars except the AA) are designed such that they will not crash into each other, however, they may crash into the AA.

When a car crosses the scope of the road, its position updates to None. Also, no new cars are deployed in the environment once the environment is initialized.
At any step, if there is a crash or if the AA has successfully crossed the scope of the road, the simulator terminates and displays the respective message.

**Getting Started**

Run all the cells in the following sections:

Environment
State
Agent
Problem

Exploring the environment through Manual Agent
To get started, run the Autonomous Driving simulator in manual control mode found in the section "Playground - Manual Agent" which uses keyboard inputs.

The commands available for manual control are:
F: to move forward
L: to move left

R: to move right
W: to wait in the same location
S: to stop the simulation (this is a command available only in manual control and is not a legal action)

Note: The action commands are case sensitive i.e., only upper-case characters will be recognized. For example, to move forward press shift+f.

Exploring the environment through Random Agent

Another option is to observe a random agent drive. Since the default agent type is random, you can simply run the cell in the section "Playground - Random Agent".

Once the simulator has started you may press enter to step through the environment.

## Input arguments to AutonomousDriving Object

| | | |
|---|---|---|
| --agent | String | Available agent types are manual, random, and expectimax. The autonomous agent (AA) is initialized as the specified agent type. |

| | | |
|---|---|---|
| --height | Integer | Specify the height of the road (discretized distance). Default is 10. |
| --width | Integer | Specify the width of the road (number of lanes). Default is 4. |
| --occupancy | Float | This argument takes a value between 0 and 0.3. Specify the weight w.r.t. the number of grid cells that must populated with other cars. For example, when it is 0, there are no other cars on the road and when it is 0.2, 20% of the cells are occupied with other cars. Default is 0.1. |
| --init | Float | This argument takes a value between 0 and 0.9. Specify the weight w.r.t. the height of the grid for initial placement of the AA. For example, when it is 0.1, the AA is initialized somewhere at the bottom of the grid and when it is 0.9, it is |

# ASU CSE 471 | Spring 2026

## Final Coding Exam Part B

*Please read the instructions carefully before beginning the exam*

**Introduction**

In this project, you will implement the agent program for a learning agent. You will test your agents on a grid-based simulation of an autonomous driving domain.

The code for this project contains the following sections, available as a notebook called FinalExam_PartB.ipynb on canvas.

[IMPORTANT] Before you begin working on your code, save the notebook file to your drive so that your progress is autosaved.

| **Sections you will edit:** | |
|---|---|
| [illegible] | [illegible] |

| | |
|---|---|
| | You may add new helper functions if needed. |
| | |
| **Sections you should read but NOT edit:** | |
| Environment | This section defines the autonomous driving environment and implements the simulator functions. You may not access the attributes and methods of this class directly. |
| State | This is an important section for your reference. This section consists of all the helper functions you will need to implement your agent program. This is your interface to the environment. |
| Agent | This section implements different agent classes such as Manual Agent and Random Agent. |

You may customize the road by using the height, width and seed arguments described above.

## Learning Agent

A learning agent in this project is an approximate Q learning agent. An approximate Q learning agent chooses an action at each choice point based on the weights it has learned over the features extracted.

Unlike the pacman project, where the features were extracted for you, in this project you will implement the get_features() function of the Learning Agent class. The get_features() function is passed a state object which is an instance of the LearningState class. The LearningState class defines a set of helper functions for you to access the attributes of the environment such as the self.road, self.cars, self.height and self.width. These attributes are your close friends when writing the get_features() function. Note that these attributes should be accessed only via the helper functions defined in the LearningState class.

```
def get_features(self, state, action):
        """
        Description
        ----------
```

Note: when writing your functions, be aware to check for terminal states or location of the AA being out of bounds, etc. wherever necessary.

Test your code by running the cell in the section "Playground - Learning Agent".

## Evaluating your Learning Agent

[IMPORTANT] First, enter the number of features you have defined in the cell in section "Grade Q3" shown below

```
num_features = None # Enter the number of features you define
q3_results = grade_q3(tests, verbose=False, features=num_features)

print('==========================')
print(f'Q3 score: {q3_results} / 10.0')
print('==========================')
```

Run the cells in the section "Evaluation" and "Test Cases". Then, run the cell in section "Grade Q3".